

2_stack.cpp

```
#include <iostream>
#include <string>
using namespace std;

#define MAX 100

class Stack {
private:
    char arr[MAX];
    int top;

public:
    Stack() {
        top = -1;
    }

    // Push operation
    void push(char value) {
        if (top == MAX - 1) {
            cout << "Stack Overflow!" << endl;
            return;
        }
        arr[++top] = value;
    }

    // Pop operation
    char pop() {
        if (top == -1) {
            return '\0';
        }
        return arr[top--];
    }

    // Peek operation
    char peek() {
        if (top == -1) {
            return '\0';
        }
        return arr[top];
    }
}
```

```

// Check if stack is empty
bool isEmpty() {
    return top == -1;
}
};

// Function to check balanced brackets
bool isBalanced(string expression) {
    Stack s;

    for (char ch : expression) {

        // Opening brackets
        if (ch == '(' || ch == '[' || ch == '{') {
            s.push(ch);
        }

        // Closing brackets
        else if (ch == ')' || ch == ']' || ch == '}') {

            if (s.isEmpty())
                return false;

            char top = s.pop();

            if ((ch == ')' && top != '(') ||
                (ch == ']' && top != '[') ||
                (ch == '}' && top != '{')) {
                return false;
            }
        }
    }

    return s.isEmpty();
}

int main() {

    // Demonstrate Stack Operations
    Stack s;

    cout << "STACK OPERATIONS" << endl;

```

```

s.push('A');
s.push('B');
s.push('C');

cout << "Top element: " << s.peek() << endl;

cout << "Popped: " << s.pop() << endl;

cout << "Top after pop: " << s.peek() << endl;

// Balanced Brackets Checker
cout << "\nBALANCED BRACKETS CHECKER" << endl;

string expression;

cout << "Enter an expression: ";
cin >> expression;

if (isBalanced(expression))
    cout << "Brackets are Balanced." << endl;
else
    cout << "Brackets are NOT Balanced." << endl;

return 0;
}

/*
Time Complexity:
Push = O(1)
Pop = O(1)
Peek = O(1)
Balanced Brackets Checker = O(n)
*/

```

Input

{[()]}

Output

STACK OPERATIONS

Top element: C
Popped: C
Top after pop: B

BALANCED BRACKETS CHECKER
Enter an expression: {[()]}
Brackets are Balanced.

Another Sample Input

{[(I)]}

Output

Brackets are NOT Balanced.

Explanation

1. Stack Class

```
class Stack
```

Creates a stack using an **array**.

2. Push

```
push(char value)
```

Adds an item to the **top** of the stack.

Example:

A
A B
A B C

Time Complexity: O(1)

3. Pop

pop()

Removes the top item.

Example:

Before:

A B C

After:

A B

Time Complexity: $O(1)$

4. Peek

peek()

Returns the top element **without removing it**.

Time Complexity: $O(1)$

5. Balanced Brackets Checker

The algorithm works like this:

- When it sees an opening bracket (, [, or {, it **pushes** it onto the stack.
- When it sees a closing bracket),], or }, it **pops** the top of the stack and checks if they match.
- If they don't match, or the stack is empty when a closing bracket appears, the expression is **not balanced**.
- At the end, if the stack is empty, the brackets are balanced.

Example:

Expression: {[()]}

Push {

Push [

Push (
Pop (matches)
Pop [matches]
Pop { matches }

Result: Balanced

Time Complexity

Operation	Complexity
Push	$O(1)$
Pop	$O(1)$
Peek	$O(1)$
Balanced Brackets Checker	$O(n)$